

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Проект: процедурный космический симулятор

Рабочее название: Project Horizon

Версия ТЗ: 0.1

Целевой жанр: одиночная приключенческая игра с процедурно создаваемой галактикой, исследованием планет, космическими полётами, добычей ресурсов, крафтом, торговлей, заданиями и строительством баз.

1. Основные архитектурные решения

1.1. Игровой движок

Использовать:

Godot Engine 4.7.1 .NET

На 31 июля 2026 года Godot 4.7.1 является актуальной стабильной версией. Движок распространяется с открытым исходным кодом и позволяет выпускать C#-проекты для Windows и Linux.

Не разрабатывать собственный 3D-движок.

Не использовать OpenGL, Vulkan или DirectX напрямую, кроме написания шейдеров через абстракции Godot. Движок сам управляет графическим API. Godot поддерживает современные рендереры на Vulkan/Direct3D 12 и режим совместимости на OpenGL.

Причины выбора Godot

1. Существенно меньшие требования к рабочим компьютерам, чем у Unreal Engine.
2. Полноценная поддержка Windows и Linux.
3. Отсутствие лицензионных платежей движку.
4. Поддержка C#.
5. Встроенная физика, сцены, навигация, анимация, звук, UI и экспорт.
6. Возможность запуска выделенного сервера без GPU.
7. Возможность модифицировать движок позднее, если появится объективная необходимость.

1.2. Рендерер

Основной режим:

Godot Mobile Renderer на Vulkan.

Несмотря на название, Mobile Renderer предназначен не только для телефонов: он может использоваться на настольных компьютерах и рассчитан на более простые сцены и повышенную производительность.

Резервный режим:

Compatibility Renderer на OpenGL 3.3.

Он предназначается для старых и слабых видеокарт. В этом режиме отключаются или упрощаются:

- сложные атмосферные эффекты;
- плотные облака;
- дальние тени;
- большое количество динамических источников света;
- сложные постэффекты;
- высокая плотность растительности.

Forward+ не использовать как базовый рендерер. Его можно добавить позднее как отдельный графический профиль «Высокий».

1.3. Языки программирования

Основной язык:

C#

На C# реализуются:

- игровая логика;
- процедурная генерация;
- управление персонажем;
- физика корабля;
- AI;
- экономика;
- задания;
- сохранения;
- сетевой код;
- инструменты разработчика;
- серверная часть.

Дополнительные языки:

Язык	Назначение
Godot Shader Language	Шейдеры поверхности планет, воды, атмосферы, растительности и интерфейсных эффектов

Язык	Назначение
SQL	Структура локальных и серверных баз данных
JSON	Описание предметов, биомов, кораблей, ресурсов, заданий и контента
Bash/PowerShell	Сборка, упаковка и автоматизация

GDScript в производственном коде не использовать. Допускаются только временные прототипы, которые затем переписываются на C#.

1.4. Целевые платформы

Обязательные:

- Windows 10 x64;
- Windows 11 x64;
- Linux x86_64.

Поддерживаемая Linux-среда:

- Ubuntu 22.04 LTS и новее;
- Linux Mint на соответствующей базе;
- SteamOS;
- другие современные дистрибутивы через Steam Linux Runtime.

Не включать в первую версию:

- macOS;
- Android;
- iOS;
- браузер;
- игровые консоли;
- VR.

2. Определение продукта

2.1. Основная концепция

Игрок управляет исследователем, способным:

1. перемещаться по поверхности планет;
2. добывать ресурсы;
3. сканировать флору, фауну и геологические объекты;
4. создавать предметы и технологии;

5. управлять космическим кораблём;
6. взлетать с поверхности;
7. перемещаться внутри звёздной системы;
8. переходить между звёздными системами;
9. взаимодействовать с NPC и фракциями;
10. торговать;
11. выполнять задания;
12. строить базы;
13. открывать планеты, системы и виды существ.

2.2. Базовый игровой цикл

Основной цикл должен выглядеть следующим образом:

Исследование → сканирование → добыча → переработка → крафт → улучшение оборудования → полёт → торговля → новое исследование.

Начальная последовательность:

1. Игрок появляется на поверхности планеты.
2. Часть оборудования неисправна.
3. Игрок находит ресурсы.
4. Ремонтирует защитный костюм и мультитул.
5. Находит корабль.
6. Ремонтирует двигатель и систему запуска.
7. Взлетает.
8. Посещает космическую станцию.
9. Получает доступ к торговле и заданиям.
10. Устанавливает межзвёздный двигатель.
11. Покидает стартовую систему.

2.3. Принцип процедурности

Процедурно создаются:

- галактическая карта;
- звёздные системы;
- звёзды;
- планеты и спутники;
- рельеф;
- биомы;
- климат;
- расположение ресурсов;
- флора;
- часть параметров фауны;

- точки интереса;
- имена;
- экономика систем;
- часть заданий.

Не создаются полностью процедурно:

- все 3D-модели;
- скелеты и анимации;
- основные сюжетные линии;
- архитектура крупных станций;
- интерфейс;
- базовые звуки;
- ключевые персонажи.

Процедурная генерация должна комбинировать подготовленные художниками модули. Попытка генерировать всё математически приведёт к низкому качеству и несовместимости моделей с анимацией.

2.4. Юридические ограничения

Запрещается копировать из No Man's Sky:

- названия;
- персонажей;
- фракции;
- сюжет;
- тексты;
- интерфейс;
- модели;
- звуки;
- музыку;
- визуальные обозначения;
- конкретный дизайн кораблей;
- исходный код;
- игровые данные.

Допускается использование общих жанровых механик: процедурные планеты, полёты, исследования, крафт и базы.

3. Масштаб первой полноценной версии

3.1. Галактика

Первая версия содержит одну галактику.

Проектная ёмкость галактики:

- до 1 048 576 потенциальных звёздных систем;
- от 1 до 8 планет в системе;
- от 0 до 4 спутников у планеты;
- генерация системы только при первом обращении;
- отсутствие предварительного хранения всех систем в базе данных.

Последующие версии могут поддерживать несколько галактик без изменения формата сохранений.

3.2. Типы звёзд

Минимальный набор:

- красный карлик;
- оранжевый карлик;
- жёлтая звезда;
- белая звезда;
- голубая звезда;
- двойная система как декоративный вариант.

Звёзды не моделируются физически. Их параметры определяют:

- цвет освещения;
- радиацию;
- температурные диапазоны планет;
- вероятность редких ресурсов;
- доступность системы для начального оборудования.

3.3. Типы планет

Для версии 1.0:

1. умеренная;
2. пустынная;
3. замёрзшая;
4. вулканическая;
5. токсичная;
6. радиоактивная;

7. безжизненная;
8. океаническая;
9. газовый гигант без возможности посадки.

Каждая планета должна иметь:

- радиус;
- силу тяжести;
- атмосферу;
- температуру;
- уровень радиации;
- токсичность;
- плотность ресурсов;
- набор биомов;
- воду или её отсутствие;
- погодные режимы;
- уровень опасности;
- собственный seed генерации.

3.4. Контент версии 1.0

Минимальное содержание:

Категория	Количество
Типы планет	9
Наземные биомы	16
Базовые ресурсы	24
Переработанные материалы	20
Рецепты	не менее 80
Технологии костюма	20
Технологии мультитула	15
Технологии корабля	25
Классы кораблей	6
Фракции	3
Основные виды NPC	3
Архетипы наземной фауны	12
Архетипы летающей фауны	4
Архетипы водной фауны	4
Модули флоры	не менее 60
Типы точек интереса	20
Типы процедурных заданий	15

Категория	Количество
Модули строительства	не менее 50
Космические станции	3 базовые архитектуры

4. Общая архитектура программы

4.1. Архитектурный стиль

Использовать многослойную архитектуру:

1. **Presentation Layer** — сцены, камеры, UI, визуальные эффекты.
2. **Application Layer** — сценарии игры и координация систем.
3. **Domain Layer** — правила мира и чистая игровая логика.
4. **Infrastructure Layer** — база данных, файлы, сеть, логирование.
5. **Tools Layer** — редакторы, генераторы и диагностика.

Критическое правило:

Процедурная генерация, экономика, предметы, задания и состояние мира не должны напрямую зависеть от Godot Node.

Это позволит:

- тестировать системы без запуска графического движка;
- использовать часть логики на сервере;
- менять UI и сцены без изменения правил игры;
- выполнять генерацию в фоновых потоках.

4.2. Структура репозитория

```

/project
  /src
    /Game.Client
      project.godot
    /Scenes
    /Scripts
    /Shaders
    /UI
    /Audio
  /Game.Domain
  /Game.Application
  /Game.WorldGen
  /Game.Persistence
  /Game.Networking
  /Game.Content
  /Game.Tools
  /server
    /Universe.Api

```

```
    /Universe.Worker
  /tests
    /Game.Domain.Tests
    /Game.WorldGen.Tests
    /Game.Persistence.Tests
    /Game.IntegrationTests
  /content
    /Items
    /Biomes
    /Planets
    /Ships
    /Species
    /Quests
    /Localization
  /art
    /Source
    /Models
    /Textures
    /Animations
    /Audio
  /build
  /docs
```

4.3. Основные модули

Game.Domain

Содержит:

- идентификаторы сущностей;
- предметы;
- инвентарь;
- рецепты;
- характеристики;
- урон;
- фракции;
- репутацию;
- экономику;
- задания;
- правила строительства;
- состояние игрока и корабля.

Не содержит ссылок на Godot.

Game.WorldGen

Содержит:

- генерацию галактики;

- генерацию систем;
- генерацию планет;
- генерацию биомов;
- генерацию рельефа;
- размещение ресурсов;
- размещение флоры;
- размещение фауны;
- размещение точек интереса;
- генерацию названий.

Game.Persistence

Содержит:

- SQLite;
- миграции;
- репозитории;
- автосохранение;
- резервные копии;
- восстановление после сбоя;
- экспорт и импорт сохранений.

Game.Application

Содержит игровые сценарии:

- начать новую игру;
- загрузить игру;
- войти в систему;
- приземлиться;
- собрать ресурс;
- создать предмет;
- купить товар;
- принять задание;
- построить модуль базы.

Game.Client

Содержит:

- Godot-сцены;
- визуальное представление;
- камеры;
- управление;

- анимацию;
- звуки;
- UI;
- адаптеры между Godot и доменной логикой.

Game.Networking

Содержит:

- клиент API;
- сетевые сообщения;
- синхронизацию;
- сериализацию;
- будущий многопользовательский режим.

4.4. Глобальные сервисы

Допускаются следующие глобальные сервисы Godot Autoload:

- Bootstrap;
- GameSession;
- SaveCoordinator;
- SceneCoordinator;
- InputRouter;
- AudioManager;
- LoadingCoordinator;
- DebugConsole.

Любые другие глобальные объекты должны быть отдельно обоснованы.

Не использовать глобальные статические переменные для хранения состояния мира.

5. Сцены игры

5.1. Основные сцены

Bootstrap
MainMenu
NewGameMenu
LoadGameMenu
GalaxyMap
StarSystemWorld
PlanetWorld
SpaceStationWorld
InteriorWorld

LoadingScreen
DebugWorld

5.2. Разделение миров

Не создавать одну сцену, содержащую всю галактику.

Одновременно загружается только необходимый уровень представления:

- галактическая карта;
- одна звёздная система;
- одна детализированная планета;
- локальная область поверхности;
- интерьер станции или базы.

Другие объекты представлены упрощёнными данными или прокси-моделями.

5.3. Переходы

Поддерживать следующие переходы:

- поверхность → атмосфера;
- атмосфера → космос;
- космос → атмосфера;
- система → гиперпереход;
- гиперпереход → новая система;
- космос → станция;
- поверхность → здание;
- здание → поверхность.

На ранних этапах допускается экран загрузки.

К версии 1.0 переход поверхность–космос должен происходить без отдельного меню.
Допускается маскировка загрузки:

- облаками;
 - плазменным эффектом;
 - тряской;
 - входом в атмосферу;
 - гиперпереходом.
-

6. Система координат

6.1. Проблема больших расстояний

Обычные 32-битные координаты теряют точность при удалении от начала координат. Это критично для космических и планетарных игр. Godot отдельно документирует ограничения точности больших миров.

6.2. Решение

Не хранить всю галактику в координатах Godot Vector3.

Использовать четыре уровня координат:

Уровень 1. Галактика

```
GalaxyId  
SectorX: int  
SectorY: int  
SectorZ: int
```

Уровень 2. Звёздная система

```
SystemId  
Double3 systemPosition
```

Double3 — собственная структура из трёх 64-битных чисел double.

Уровень 3. Планета

```
PlanetId  
PlanetFixedPosition  
PlanetRotation
```

Уровень 4. Локальная сцена

```
CellX: long  
CellY: long  
CellZ: long  
LocalPosition: Vector3
```

Размер локальной ячейки:

4096 метров.

Один юнит Godot в локальной сцене:

1 метр.

При удалении игрока от локального начала координат более чем на 2048 метров выполняется перенос начала координат — floating origin.

Визуально игрок остаётся около (0, 0, 0), а логические координаты корректируются.

7. Идентификация и seed-иерархия

7.1. Seed мира

При создании игры генерируется:

UniverseSeed: unsigned 64-bit integer

Пользователь может:

- использовать случайный seed;
- ввести seed вручную;
- поделиться seed с другим игроком.

7.2. Производные seed

Каждый объект получает seed через устойчивое хеширование:

```
UniverseSeed
  → GalaxySeed
    → SectorSeed
      → SystemSeed
        → StarSeed
          → PlanetSeed
            → BiomeSeed
              → RegionSeed
                → FloraSeed
                  → FaunaSeed
                    → PoiSeed
```

Запрещается использовать последовательный глобальный генератор случайных чисел для всего мира.

Изменение порядка генерации одного объекта не должно изменять остальные объекты.

7.3. Версия генератора

Каждое сохранение содержит:

```
generator_version
content_version
schema_version
```

После выпуска игры нельзя молча менять алгоритм генерации существующих миров.

Варианты изменения:

1. сохранить старый генератор для старых сохранений;
 2. выполнить контролируемую миграцию;
 3. создать новую версию вселенной.
-

8. Генерация галактики

8.1. Структура

Галактика представляется разреженной трёхмерной сеткой секторов.

Не каждый сектор содержит систему.

Для каждого сектора детерминированно определяются:

- наличие системы;
- тип звезды;
- число планет;
- опасность;
- экономика;
- фракция;
- редкость;
- соседние маршруты.

8.2. Межзвёздные маршруты

Игрок не должен иметь возможность мгновенно перемещаться в любую систему.

Для перехода учитываются:

- дальность двигателя;
- запас топлива;
- тип звезды;
- известность системы;
- ограничения задания;
- специальные технологии.

Маршрут рассчитывается графовым алгоритмом A*.

8.3. Карта галактики

Карта должна поддерживать:

- вращение;
- масштабирование;
- выбор системы;
- построение маршрута;
- фильтрацию по экономике;
- фильтрацию по фракции;
- фильтрацию по звезде;
- отображение посещённых систем;

- отображение активных заданий;
- постановку пользовательской метки.

На карте не создаются отдельные Node3D для миллиона звёзд. Используется пакетный GPU-рендеринг.

9. Генерация планет

9.1. Геометрическая модель

Планета строится как **cube sphere**:

1. создаётся куб;
2. каждая из шести граней делится на квадратные участки;
3. вершины проецируются на сферу;
4. к радиусу добавляется высота рельефа.

Преимущества:

- удобное разбиение на квадратные чанки;
- контролируемый LOD;
- отсутствие полюсов с чрезмерной плотностью вершин;
- возможность независимой генерации участков.

9.2. Радиусы

Игровые радиусы планет:

- малые: 20–30 км;
- средние: 30–50 км;
- крупные: 50–80 км.

Это не астрономический масштаб. Расстояния намеренно сжаты.

9.3. Quadtree LOD

Каждая грань cube sphere управляется quadtree.

Параметры чанка:

- сетка: 33 × 33 вершины;
- один чанк: 2048 треугольников без учёта соединительных элементов;
- 10–12 уровней детализации;
- повышенная детализация возле игрока;
- пониженная детализация на горизонте;

- выгрузка невидимых чанков;
- отдельное создание коллизии.

Для устранения щелей использовать:

- skirts;
- согласование соседних LOD;
- geomorphing;
- одинаковое вычисление высот на общей границе.

9.4. Формирование высоты

Высота определяется комбинацией:

```
continental_mask  
+ hills_noise  
+ ridge_noise  
+ mountain_mask  
+ crater_mask  
+ erosion_approximation  
+ biome_modifier
```

Не выполнять полноценную гидравлическую эрозию во время игры.

Использовать:

- OpenSimplex2;
- Perlin;
- Cellular noise;
- domain warp;
- кривые преобразования;
- маски широты и высоты.

Godot содержит FastNoiseLite с несколькими видами шума, включая Cellular, Perlin и Value.

9.5. Биомы

Биом определяется по:

- широте;
- высоте;
- температуре;
- влажности;
- типу атмосферы;
- расстоянию до воды;
- локальному шуму;
- параметрам планеты.

На одной планете допускается до восьми активных биомов.

Примеры:

- равнина;
- лес;
- болото;
- пустыня;
- горы;
- тундра;
- ледник;
- вулканическое поле;
- каменистое плато;
- радиоактивная пустошь;
- токсичные заросли;
- прибрежная зона.

9.6. Вода

Вода реализуется как сферическая поверхность фиксированного уровня.

Поддерживаются:

- океаны;
- озёра как локальные упрощённые объекты;
- отражение неба;
- глубинное затемнение;
- волны через шейдер;
- подводный постэффект;
- плавание.

Не реализуется физическая симуляция жидкости.

9.7. Атмосфера

Использовать упрощённый атмосферный шейдер:

- сферическая оболочка;
- градиент рассеяния;
- цвет, зависящий от направления на звезду;
- усиление на горизонте;
- плотность атмосферы;
- закатный цвет.

Не использовать дорогой многократный объёмный ray marching на слабом профиле.

9.8. Облака

В версии 1.0:

- один или два сферических облачных слоя;
- прокручиваемые noise-текстуры;
- изменение плотности;
- затенение поверхности в упрощённом виде.

Полноценные объёмные облака не являются обязательными.

9.9. Пещеры и изменение рельефа

В версии 1.0:

- глобальные процедурные пещеры отсутствуют;
- допускаются отдельные пещерные prefab-объекты;
- пользователь не изменяет планетарный рельеф;
- ресурсы добываются из объектов и месторождений.

Локальная воксельная модификация рельефа рассматривается только после выпуска стабильной версии основной игры.

10. Стриминг мира

10.1. Активная зона

В полной детализации поддерживается область радиусом:

- пешком: 1,5–2,5 км;
- при наземном транспорте: 4–6 км;
- при атмосферном полёте: 10–20 км.

За пределами активной зоны используются упрощённые чанки.

10.2. Очередь генерации

Чанки имеют приоритеты:

1. чанк под игроком;
2. чанки направления движения;
3. коллизионные чанки;
4. видимые чанки;
5. дальние чанки;
6. предварительная генерация.

10.3. Фоновые потоки

В фоне разрешается:

- вычислять высоты;
- генерировать массивы вершин;
- размещать объекты;
- читать файлы;
- выполнять SQL-запросы;
- рассчитывать задания и экономику.

В основном потоке выполняется:

- создание Node;
- добавление Mesh;
- добавление CollisionShape;
- изменение SceneTree;
- передача данных GPU.

Godot поддерживает многопоточность не во всех частях API, поэтому фоновые задачи должны возвращать обычные массивы данных, а создание объектов сцены должно выполняться контролируемо.

Количество рабочих задач:

```
max(1, min(4, logical_processor_count - 2))
```

10.4. Ограничение задержек

Генерация не должна блокировать основной поток дольше:

- 2 мс в обычном кадре;
- 5 мс при принудительной подгрузке;
- 10 мс только на экране загрузки.

Все задания генерации должны поддерживать отмену.

11. Растительность и природные объекты

11.1. Размещение

Объекты размещаются детерминированно по region seed.

Учитываются:

- биом;

- высота;
- уклон;
- влажность;
- температура;
- расстояние до воды;
- плотность;
- минимальное расстояние между объектами.

11.2. Рендеринг

Для травы, камней и повторяющихся растений использовать MultiMesh и GPU instancing. Godot MultiMesh предназначен для высокопроизводительного вывода большого количества экземпляров одной геометрии.

Один MultiMesh не должен покрывать всю планету.

Разбиение:

- один набор MultiMesh на регион;
- один набор на тип объекта;
- отдельные уровни детализации;
- отключение мелких объектов по расстоянию.

11.3. Интерактивные объекты

Объект превращается из GPU-экземпляра в полноценную игровую сущность только когда:

- находится рядом с игроком;
- сканируется;
- получает урон;
- добывается;
- участвует в задании.

После удаления игрока объект возвращается в упрощённое состояние.

12. Фауна

12.1. Принцип генерации

Фауна создаётся из совместимых модулей.

Базовые планы тела:

- двуногое;

- четвероногое;
- шестиногое;
- летающее;
- водное;
- ползающее.

Каждый план тела имеет фиксированный скелет.

Процедурно выбираются:

- голова;
- корпус;
- конечности;
- хвост;
- рога;
- панцирь;
- размеры;
- пропорции в допустимых пределах;
- материал;
- окраска;
- поведение;
- рацион;
- активность;
- уровень агрессии.

Запрещается соединять модули разных скелетов без подготовленной системы ретаргетинга.

12.2. Поведение

Архитектура AI:

- иерархический конечный автомат;
- utility scoring для выбора действия;
- steering для движения;
- navmesh для наземных существ;
- упрощённый boids-подход для стаи.

Состояния:

Idle
Wander
Graze
Drink
Sleep
Investigate

Flee
Threaten
Attack
ReturnToTerritory
FollowGroup

12.3. Частота обновления

- ближайшие существа: 10 раз в секунду;
- средняя дистанция: 2–5 раз в секунду;
- дальние существа: статистическая симуляция;
- визуальная интерполяция: каждый кадр.

12.4. Лимиты

На одной локальной области:

- до 20 полноценно активных существ;
 - до 80 упрощённых существ;
 - остальные представлены данными популяции.
-

13. Персонаж игрока

13.1. Режим управления

Базовый режим:

вид от первого лица.

Вид от третьего лица допускается после завершения основного игрового цикла.

13.2. Возможности

Персонаж должен уметь:

- ходить;
- бегать;
- прыгать;
- приседать;
- использовать реактивный ранец;
- плавать;
- взаимодействовать;
- сканировать;
- добывать ресурсы;
- стрелять;

- получать урон;
- использовать предметы;
- входить в корабль;
- строить.

13.3. Характеристики

Health
Shield
Stamina
LifeSupport
HazardProtection
TemperatureProtection
RadiationProtection
ToxicProtection
Oxygen
JetpackEnergy

13.4. Планетарная гравитация

Гравитация направляется не по глобальной оси вниз, а к центру текущей планеты.

`gravityDirection = normalize(planetCenter - playerPosition)`

Ориентация персонажа должна плавно согласовываться с нормалью поверхности.

13.5. Мультитул

Мультитул совмещает:

- сканер;
- добывающий луч;
- оружие;
- анализатор;
- инструмент ремонта.

Поддерживается установка модулей улучшения.

14. Космический корабль

14.1. Управление

Корабль использует упрощённую аркадную физику.

В космосе:

- тяга вперёд и назад;

- рыскание;
- тангаж;
- крен;
- боковые импульсные двигатели;
- форсаж;
- торможение;
- автоматическая стабилизация.

В атмосфере:

- подъёмная сила упрощена;
- минимальная скорость;
- сопротивление;
- ограничение вертикального набора;
- автоматическое предотвращение грубого столкновения с поверхностью.

Не реализуется полноценная аэродинамическая модель.

14.2. Классы кораблей

1. исследовательский;
2. грузовой;
3. истребитель;
4. универсальный;
5. экзотический;
6. тяжёлый экспедиционный.

Параметры:

Hull
 Shield
 CargoCapacity
 FuelCapacity
 Acceleration
 MaxSpeed
 Maneuverability
 WeaponSlots
 TechnologySlots
 HyperdriveRange
 AtmosphericEfficiency

14.3. Повреждения

Отдельно повреждаются:

- корпус;
- щит;
- двигатель;

- импульсная система;
- гипердвигатель;
- оружие;
- система посадки.

14.4. Посадка

Посадка включает:

1. поиск допустимой поверхности;
 2. проверку уклона;
 3. проверку препятствий;
 4. резервирование точки;
 5. автоматическое выравнивание;
 6. отключение основной физики;
 7. установку корабля на опоры.
-

15. Космическая симуляция

15.1. Модель системы

Не использовать реальную небесную механику.

Планеты могут:

- двигаться по заранее рассчитанным орбитам;
- либо визуально вращаться вокруг звезды с сильно замедленным временем.

Гравитационное взаимодействие планет не рассчитывается.

15.2. Уровни представления

На большой дистанции:

- планета — сфера низкой детализации;
- атмосфера — оболочка;
- облака — один слой;
- станции — прокси-модели;
- корабли — маркеры или упрощённые модели.

При приближении к планете загружается PlanetRuntime.

15.3. Активация поверхности

Порядок:

1. корабль входит в область планеты;
2. загружаются параметры планеты;
3. строятся дальние LOD;
4. определяется точка под кораблём;
5. создаются чанки поверхности;
6. включается атмосферная физика;
7. создаются коллизии;
8. загружаются объекты региона.

Одновременно детально активна только одна планета.

16. NPC и фракции

16.1. Фракции

Для версии 1.0:

1. торговая;
2. научная;
3. военная.

У каждой фракции имеются:

- репутация;
- экономические интересы;
- предпочтительные товары;
- типы заданий;
- визуальный стиль;
- набор имён;
- отношения с другими фракциями.

16.2. NPC

Типы NPC:

- торговец;
- техник;
- пилот;
- учёный;
- охранник;
- представитель гильдии;
- случайный путешественник;
- противник.

16.3. Диалоги

Использовать шаблонную диалоговую систему.

Диалог состоит из:

- идентификатора;
- условий;
- локализованных реплик;
- вариантов ответа;
- последствий;
- изменения репутации;
- запуска задания;
- торгового действия.

Генеративный AI в работающей игре не требуется.

17. Инвентарь, ресурсы и крафт

17.1. Инвентари

Отдельные контейнеры:

- костюм;
- мультитул;
- корабль;
- хранилище базы;
- торговый терминал;
- временный контейнер добычи.

17.2. Предмет

ItemId
DefinitionId
Quantity
Quality
Durability
CustomData
ContainerId
SlotIndex

17.3. Определение предмета

Статические параметры:

DefinitionId
LocalizationKey
Category
MaxStack
BasePrice
Mass
Rarity
Icon
WorldModel
Tags

17.4. Крафт

Рецепт содержит:

RecipeId
Inputs[]
Outputs[]
RequiredTechnology
RequiredStation
CraftTime

Рецепты должны быть data-driven и храниться в контентных JSON-файлах.

17.5. Переработка

Отдельные устройства:

- переносной переработчик;
- стационарный переработчик;
- промышленный модуль базы.

18. Экономика

18.1. Типы экономики

Минимальный набор:

- добывающая;
- промышленная;
- технологическая;
- торговая;
- научная;
- военная.

18.2. Цена

FinalPrice =
BasePrice
× SystemEconomyModifier
× SupplyDemandModifier
× FactionModifier
× ReputationModifier
× RandomDailyModifier

18.3. Ограничения

Не симулировать все сделки во всей галактике.

Состояние экономики рассчитывается:

- при первом посещении;
- при возвращении;
- при совершении сделки;
- после значительного игрового времени.

Оффлайн-изменение вычисляется формулой по разнице времени.

19. Задания

19.1. Типы задач

VisitLocation
ScanObject
ScanSpecies
CollectResource
CraftItem
DeliverItem
RepairObject
DefeatTarget
ProtectTarget
BuildModule
TradeItem
FindSignal
ExplorePlanet
ExploreSystem
ReturnToNpc

19.2. Структура задания

Задание представляется графом состояний.

QuestDefinition
QuestInstance

QuestNode
QuestCondition
QuestAction
QuestReward

19.3. Валидация

Генератор не должен создавать невыполнимые задания.

До выдачи проверяется:

- доступность системы;
- наличие требуемого ресурса;
- возможность посадки;
- соответствие уровню оборудования;
- существование цели;
- наличие места в инвентаре;
- отсутствие циклической зависимости.

19.4. Сюжет

Основная сюжетная линия создаётся вручную.

Процедурно генерируются только повторяемые задания.

20. Строительство баз

20.1. Принцип

Использовать модульное строительство с привязкой к точкам соединения.

Категории:

- фундамент;
- пол;
- стена;
- крыша;
- коридор;
- дверь;
- окно;
- лестница;
- помещение;
- генератор;
- аккумулятор;

- переработчик;
- хранилище;
- посадочная площадка;
- терминал;
- декоративные элементы.

20.2. Ограничения

На одну базу:

- не более 500 строительных модулей;
- не более 100 интерактивных устройств;
- не более 200 активных физических объектов;
- не более 20 источников динамического света.

20.3. Сети базы

Поддержать:

- электрическую сеть;
- генераторы;
- аккумуляторы;
- потребителей;
- включение и отключение устройств.

Сеть представляется графом.

20.4. Рельеф

В версии 1.0 база не изменяет геометрию планеты.

Допускаются:

- регулируемые опоры;
- платформы;
- автоматическое выравнивание;
- скрывание мелких растений под постройкой.

21. Точки интереса

Типы:

- аварийный маяк;
- обломки корабля;
- заброшенная база;

- торговый пост;
- научная станция;
- древний объект;
- ресурсное месторождение;
- пещерный вход;
- ретранслятор;
- производственный объект;
- база пиратов;
- посадочная площадка;
- монолит;
- капсула улучшения;
- грузовой контейнер.

Размещение учитывает:

- расстояние между объектами;
 - тип биома;
 - уклон;
 - высоту;
 - расстояние до воды;
 - опасность;
 - редкость;
 - задания.
-

22. Локальная база данных

22.1. Выбор БД

Использовать:

SQLite через Microsoft.Data.Sqlite.

SQLite является встраиваемой базой данных, работающей внутри приложения и не требующей установки отдельного сервера. Microsoft.Data.Sqlite предоставляет легковесный ADO.NET-интерфейс для C#.

Не использовать:

- MySQL;
- PostgreSQL;
- MongoDB;
- отдельный локальный сервер БД;
- Entity Framework в клиенте.

SQL-запросы и миграции должны быть явными.

22.2. Файлы

```
/user_data
  /settings.json
  /profiles
    /profile_<id>
      profile.db
      save_1.db
      save_1.backup.db
      screenshots/
      logs/
```

Один игровой слот — одна SQLite-база.

22.3. Режим работы

При открытии базы:

```
PRAGMA journal_mode = WAL;
PRAGMA foreign_keys = ON;
PRAGMA synchronous = NORMAL;
PRAGMA busy_timeout = 5000;
```

WAL использует журнал предварительной записи и сохраняется как режим базы между подключениями.

Запись в базу выполняется через одну последовательную очередь.

22.4. Таблицы

Служебные

```
schema_migrations
save_meta
save_settings
```

Игрок

```
player_state
player_reputation
player_technology
player_statistics
```

Инвентарь

```
containers
inventory_items
installed_modules
```

Корабли

```
ships
ship_damage
```

ship_technology
ship_customization

Мир

visited_systems
visited_planets
discoveries
named_objects
entity_deltas
region_deltas
depleted_resources
opened_containers
destroyed_objects

Базы

bases
base_modules
base_connections
base_storage
base_power_state

Задания

quest_instances
quest_node_states
quest_variables

Экономика

station_economy
trade_history
station_inventory

22.5. Что не хранится в базе

Не хранить:

- полный рельеф планеты;
- все деревья;
- все камни;
- все животные;
- все звёздные системы;
- полные меши;
- текстуры;
- статические определения предметов.

Они восстанавливаются из seed и версии генератора.

Хранятся только изменения относительно процедурного состояния.

22.6. Region delta

Пример содержимого:

```
RegionKey  
RemovedResourceIds[]  
OpenedContainerIds[]  
PlacedBaseModuleIds[]  
DestroyedPoiIds[]  
CustomEntityStates[]  
LastVisitedTime
```

Большие наборы изменений сохраняются в сжатом бинарном BLOB.

22.7. Пространственный индекс

Для большого числа пользовательских баз и объектов допускается SQLite R-Tree. Этот индекс предназначен для пространственных диапазонных запросов и может использоваться в трёх измерениях.

Использование R-Tree необязательно до появления реальной нагрузки.

22.8. Автосохранение

Автосохранение выполняется:

- каждые 60 секунд;
- после посадки;
- после взлёта;
- после гиперперехода;
- после завершения задания;
- после покупки корабля;
- после изменения базы;
- при корректном выходе.

Сохранение выполняется транзакцией.

22.9. Восстановление

Требования:

- последняя корректная резервная копия;
 - контроль целостности;
 - атомарная замена файла;
 - журнал ошибки;
 - возможность загрузить предыдущую копию;
 - запрет уничтожения единственного исправного сохранения.
-

23. Статические игровые данные

Статические определения не хранятся в SQLite.

Использовать JSON:

```
items.json
recipes.json
planet_archetypes.json
biomes.json
resources.json
ships.json
technologies.json
factions.json
quest_templates.json
building_modules.json
```

Каждое определение имеет стабильный строковый идентификатор:

```
resource.iron
resource.carbon
ship.engine.basic
planet.archetype.frozen
biome.lush_forest
quest.delivery.basic
```

Не использовать порядковый номер массива как постоянный ID.

Все JSON-файлы проходят проверку схемы при сборке.

24. Серверная платформа

24.1. Необходимость сервера

Одиночная игра должна полностью работать без интернета.

Сервер не должен быть обязательным для:

- создания мира;
- загрузки сохранения;
- исследования;
- торговли;
- заданий;
- строительства;
- процедурной генерации.

24.2. Рекомендуемые серверные функции

Сервер целесообразно использовать для:

1. учётных записей;
2. облачных сохранений;
3. реестра открытий;
4. загрузки названий систем и планет;
5. статистики;
6. добровольной телеметрии;
7. отчётов о сбоях;
8. доставки конфигураций;
9. проверки актуальной версии;
10. будущего поиска многопользовательской сессии.

24.3. Серверный стек

ASP.NET Core Web API

PostgreSQL

S3-compatible object storage

Docker Compose (при необходимости)

Linux

Redis не использовать, пока не возникнет измеренная необходимость.

На первом этапе сервер должен быть модульным монолитом, а не набором микросервисов.

24.4. Серверные модули

Identity

CloudSaves

Discoveries

ContentManifest

Telemetry

CrashReports

Matchmaking

Administration

24.5. Облачные сохранения

В PostgreSQL хранятся:

- пользователь;
- слот;
- версия;
- контрольная сумма;
- дата изменения;
- размер;

- ссылка на объект.

Сам файл сохранения хранится в объектном хранилище.

Не пытаться синхронизировать отдельные строки SQLite между устройствами. Синхронизируется целый зашифрованный или сжатый snapshot.

24.6. Реестр открытий

Сервер хранит:

UniverseSeed
ObjectId
ObjectType
OriginalGeneratedName
PlayerAssignedName
DiscovererId
DiscoveredAt
ModerationStatus

Пользовательские названия проходят:

- ограничение длины;
 - фильтрацию;
 - нормализацию;
 - модерацию;
 - проверку конфликтов.
-

25. Будущий многопользовательский режим

25.1. Этап внедрения

Многопользовательский режим не разрабатывается до готовности стабильной одиночной вертикали.

Первая сетевая версия:

- 2–4 игрока;
- кооператив;
- одна система;
- один сервер-владелец сессии либо выделенный сервер;
- общие базы;
- совместные задания.

25.2. Выделенный сервер

Использовать Godot headless.

Godot поддерживает экспорт выделенных серверов и запуск без графического устройства через headless-режим.

25.3. Транспорт

Для игрового состояния:

ENet/UDP через Godot Multiplayer API.

ENet в Godot работает поверх UDP и может создавать клиентские и серверные соединения.

REST API не использовать для синхронизации движения.

25.4. Авторитетность

Сервер является авторитетным для:

- позиции;
- урона;
- инвентаря;
- добычи;
- торговли;
- крафта;
- строительства;
- наград;
- заданий.

Клиент выполняет:

- prediction;
- interpolation;
- визуальные эффекты;
- предварительный UI-отклик.

25.5. Ограничение области синхронизации

Клиент получает только объекты:

- текущей системы;
- текущей планеты;
- ближайших регионов;
- активной группы;
- текущего задания.

Вся галактика по сети не передаётся.

Высокоуровневый Godot Multiplayer API следует использовать только между Godot-клиентом и Godot-сервером: его внутренний протокол не предназначен для произвольных сторонних серверов.

26. Графика

26.1. Визуальный стиль

Для первой версии использовать:

стилизованный PBR средней детализации.

Не пытаться достигать фотореализма.

Требования:

- читаемые силуэты;
- ограниченное количество материалов;
- единая цветовая система;
- умеренная геометрическая детализация;
- акцент на разнообразии планет;
- отсутствие дорогих микродеталей.

26.2. Размеры текстур

Объект	Максимальный размер
Игровой персонаж	2048 × 2048
Крупный корабль	2048 × 2048
Обычный NPC	1024–2048
Крупная постройка	2048
Обычный объект	512–1024
Растение	512–1024
UI-иконка	128–512
Тайловая поверхность	1024–2048

Использовать texture atlases и повторно используемые материалы.

26.3. Освещение

На поверхности:

- один основной направленный источник — звезда;
- окружающее освещение;
- ограниченное число локальных источников;

- тени только у значимых объектов;
- упрощённое освещение дальних объектов.

В интерьерах:

- запечённое освещение;
- локальные динамические источники;
- ограниченные тени.

26.4. Графические профили

Low

- минимальная растительность;
- дальность поверхности 50–60%;
- низкое качество теней;
- облака низкого качества;
- упрощённая вода;
- минимум постэффектов;
- ограниченные частицы.

Medium

- стандартная плотность;
- полная основная дальность;
- средние тени;
- атмосферные эффекты;
- нормальное качество воды.

High

- повышенная плотность;
- улучшенные тени;
- повышенная дальность;
- улучшенная атмосфера;
- дополнительные частицы.

Compatibility

- OpenGL 3.3;
 - упрощённые шейдеры;
 - отсутствие тяжёлых эффектов;
 - уменьшенная плотность мира;
 - отдельный набор параметров.
-

27. Производительность

27.1. Целевые показатели

Основная цель:

- 1920 × 1080;
- 60 FPS на среднем профиле;
- 30 FPS на минимально поддерживаемой машине.

27.2. Бюджет кадра

Для 60 FPS:

CPU frame: ≤ 16,6 мс

GPU frame: ≤ 16,6 мс

Для 30 FPS:

CPU frame: ≤ 33,3 мс

GPU frame: ≤ 33,3 мс

27.3. Бюджеты сцены

На профиле Medium:

- до 1500 draw calls;
- до 2 миллионов видимых треугольников на открытой местности;
- до 500 активных физических тел;
- до 20 полноценных AI;
- до 80 упрощённых AI;
- до 4 ГБ видеопамяти;
- до 6 ГБ оперативной памяти игры.

На Low значения уменьшаются минимум на 30%.

27.4. Сборка мусора

В обычном игровом кадре:

- избегать LINQ в горячих циклах;
- не создавать временные списки;
- использовать object pooling;
- переиспользовать буферы;
- не создавать строки каждый кадр;
- не сериализовать JSON каждый кадр.

Целевой объём управляемых выделений:

менее 256 КБ на кадр, исключая загрузочные операции.

27.5. Профилирование

Оптимизация выполняется только после измерений.

Godot имеет встроенный профилировщик CPU; документация прямо рекомендует сначала определить фактические узкие места.

28. Минимальные системные требования

28.1. Компьютер разработчика

Минимально:

- 64-битный четырёхъядерный CPU;
- 16 ГБ RAM;
- SSD;
- GPU с Vulkan;
- 4 ГБ видеопамати либо современная интегрированная графика;
- два монитора желательны, но не обязательны.

Рекомендуется:

- 6–8 ядер;
- 32 ГБ RAM;
- GPU уровня GTX 1060 / RX 580 или выше;
- NVMe SSD.

Godot Compatibility Renderer может работать на интегрированной графике с OpenGL 3.3, но основной проект должен тестироваться также на Vulkan.

28.2. Минимальная конфигурация игрока

- Windows 10 x64 или Linux x86_64;
- четырёхъядерный CPU;
- 8 ГБ RAM;
- 4 ГБ видеопамати либо поддерживаемая интегрированная графика;
- Vulkan либо OpenGL 3.3 для Compatibility-сборки;
- SSD;
- 20 ГБ свободного места.

28.3. Рекомендуемая конфигурация

- шестиядерный CPU;

- 16 ГБ RAM;
 - GPU с 6 ГБ видеопамяти;
 - SSD;
 - 30 ГБ свободного места.
-

29. Физика

29.1. Частота

Основная физическая симуляция:

60 Гц.

Экономика и фоновые системы не привязываются к физическому такту.

29.2. Слои коллизий

Пример:

- 1 Terrain
- 2 Player
- 3 NPC
- 4 Fauna
- 5 Ship
- 6 Projectile
- 7 Resource
- 8 Building
- 9 Vehicle
- 10 Trigger
- 11 Water
- 12 NavigationObstacle

29.3. Коллизии поверхности

Коллизии создаются только возле игрока.

Для дальнего корабля столкновение с планетой проверяется через функцию высоты, а не через миллионы треугольников.

29.4. Снаряды

Для быстрых снарядов использовать:

- raycast;
- shape cast;
- непрерывную проверку;
- object pooling.

Не создавать новый Node для каждой частицы лучевого оружия.

30. Навигация

30.1. Наземные NPC

Использовать Godot NavigationServer3D и локальные NavigationRegion3D. Godot поддерживает 3D navigation maps, regions, agents, obstacles и links.

Не создавать navmesh для всей планеты.

Navmesh создаётся по тайлам вокруг активной области.

30.2. Летающие существа

Использовать:

- steering;
- избегание сферических препятствий;
- ограничение высоты;
- точки интереса;
- локальную сетку пространства.

30.3. Космические корабли NPC

Использовать:

- steering;
 - pursuit;
 - evade;
 - arrive;
 - formation;
 - avoidance;
 - боевые состояния.
-

31. Интерфейс

31.1. Обязательные экраны

- главное меню;
- новая игра;
- загрузка;
- настройки;

- пауза;
- HUD;
- инвентарь;
- технологии;
- журнал заданий;
- карта планеты;
- карта системы;
- карта галактики;
- торговля;
- строительство;
- каталог открытий;
- управление кораблями;
- экран смерти.

31.2. Управление

Обязательно:

- клавиатура и мышь;
- переназначение клавиш;
- регулировка чувствительности;
- инверсия осей;
- поддержка стандартного геймпада;
- отдельное управление пешком и в корабле.

31.3. Локализация

Первая версия:

- русский;
- английский.

Запрещены захардкоженные пользовательские строки.

Все строки имеют localization key.

31.4. Доступность

Минимально:

- изменение размера интерфейса;
- субтитры;
- отключение тряски камеры;
- отключение motion blur;
- настройка поля зрения;

- цветовые обозначения, дублируемые формой или иконкой;
 - отдельная громкость музыки, эффектов и речи.
-

32. Звук

Использовать звуковые шины:

Master
Music
Ambient
SFX
UI
Voice
Vehicle
Weather

Требования:

- 3D-позиционирование;
 - затухание;
 - ограничение одновременных звуков;
 - пулы аудиоплееров;
 - разные окружения для атмосферы, вакуума, помещений и воды;
 - отсутствие обычного внешнего звука оружия в вакууме, кроме звука внутри корабля и условных интерфейсных сигналов.
-

33. Контентный конвейер

33.1. 3D-модели

Основной редактор:

Blender.

Формат передачи в Godot:

glTF 2.0 / GLB.

33.2. Правила моделей

- единицы Blender: метры;
- оси согласовать с Godot;
- применить scale перед экспортом;
- понятные имена костей;
- отдельные LOD-модели;

- минимальное количество материалов;
- отсутствие неиспользуемых вершин;
- проверка нормалей;
- коллизии создаются отдельно;
- точки крепления задаются через маркеры.

33.3. Именование

CHR_Player_Base
NPC_Trader_A
SHP_Explorer_01
BLD_Wall_Metal_01
FLR_Tree_Lush_01
FNA_Quadruped_Head_03
MAT_Metal_Painted
TEX_Metal_BaseColor
TEX_Metal_Normal

33.4. Исходники

Исходные .blend, несжатые текстуры и звуки не должны попадать в финальный игровой пакет.

34. Инструменты разработчика

Обязательные внутренние инструменты:

34.1. Seed Explorer

Позволяет:

- вводить seed;
- просматривать систему;
- генерировать список планет;
- менять параметры;
- копировать идентификаторы;
- экспортировать отчёт.

34.2. Planet Preview

Позволяет:

- просматривать планету;
- перемещать камеру;
- менять LOD;

- включать сетку чанков;
- показывать биомы;
- показывать высоту;
- показывать плотность ресурсов;
- измерять время генерации.

34.3. Chunk Profiler

Показывает:

- количество загруженных чанков;
- состояние очереди;
- время CPU;
- время загрузки GPU;
- объём памяти;
- число вершин;
- число коллизий;
- отменённые задания.

34.4. Save Inspector

Позволяет:

- открыть сохранение;
- показать версию;
- проверить целостность;
- показать игрока;
- показать корабли;
- показать посещённые системы;
- экспортировать таблицы;
- применить миграцию к копии.

34.5. Debug Console

Команды:

```
teleport  
spawn  
give  
damage  
heal  
set_time  
set_weather  
load_system  
load_planet  
show_chunks
```

show_navmesh
show_ai
profile_worldgen
save
reload_content

Команды отключаются в публичной сборке либо требуют режима разработчика.

35. Логирование и диагностика

Использовать структурированные категории:

BOOT
CONTENT
WORLDGEN
STREAMING
DATABASE
SAVE
PLAYER
SHIP
AI
QUEST
NETWORK
SERVER
PERFORMANCE
ERROR

Лог содержит:

- UTC-время;
- уровень;
- категорию;
- идентификатор сессии;
- сообщение;
- исключение;
- системную информацию;
- текущую сцену;
- seed мира;
- текущий объект мира.

Не записывать в лог пароли, токены и персональные данные.

36. Тестирование

36.1. Модульные тесты

Обязательны для:

- seed-иерархии;
- генерации ID;
- инвентаря;
- рецептов;
- экономики;
- заданий;
- миграций;
- расчёта характеристик;
- сериализации;
- координатных преобразований.

36.2. Golden seed tests

Создать набор фиксированных seed.

Для каждого проверяются:

- тип звезды;
- количество планет;
- параметры планет;
- контрольные высоты;
- расположение тестовых объектов;
- контрольная сумма результата.

Изменение результата требует сознательного повышения `generator_version`.

36.3. Тесты сохранений

Проверить:

- нормальное сохранение;
- выключение во время сохранения;
- повреждённую основную базу;
- восстановление backup;
- миграцию старой версии;
- неизвестный предмет;
- удалённую технологию;
- изменение контентного каталога.

36.4. Нагрузочные тесты

Сценарии:

- два часа полёта;
- восемь часов автоматического перемещения;
- 100 последовательных посадок;
- 100 гиперпереходов;
- строительство базы из 500 модулей;
- инвентарь с 10 000 записями;
- посещение 1000 систем;
- база сохранения размером 1 ГБ;
- многократное аварийное завершение.

36.5. Покрытие

Целевое покрытие тестами:

- Domain: не менее 80%;
 - WorldGen: не менее 70%;
 - Persistence: не менее 80%;
 - визуальные компоненты: smoke tests.
-

37. Сборка и контроль версий

37.1. Контроль версий

Использовать Git.

Git LFS использовать для:

- .blend;
- больших текстур;
- звуков;
- видео;
- крупных бинарных файлов.

Не хранить в Git:

- импортированный кеш Godot;
- временные сборки;
- пользовательские настройки IDE;
- локальные базы;
- логи.

37.2. Ветки

```
main  
develop  
feature/*  
fix/*  
release/*
```

main всегда должен собираться.

37.3. CI

Для каждого merge request:

1. восстановление зависимостей;
2. сборка C#;
3. unit tests;
4. проверка JSON;
5. проверка миграций;
6. экспорт Windows debug;
7. экспорт Linux debug.

Для release:

- Windows release;
 - Linux release;
 - архив символов;
 - контрольные суммы;
 - версия;
 - changelog.
-

38. Правила программирования

Обязательные:

- Nullable включён;
- предупреждения компилятора в CI считаются ошибками;
- публичные интерфейсы документируются;
- зависимости передаются явно;
- асинхронные операции принимают CancellationToken;
- доступ к SQLite параметризован;
- исключения не подавляются;
- игровые ID не зависят от имени класса;
- сериализуемые структуры версионизируются;

- запрещены циклические зависимости проектов;
- Godot Node не используется как доменная модель;
- запрещена логика предметов непосредственно в UI;
- запрещён SQL внутри сцен;
- запрещена генерация мира непосредственно в `_Process`.

38.1. Частоты систем

Rendering: каждый кадр

Physics: 60 Гц

Player controller: 60 Гц

Nearby AI: 10 Гц

Distant AI: 2 Гц

Economy UI: по событию

Background economy: 0,2–1 Гц

Save queue: по событиям

Telemetry: пакетами

38.2. События

Использовать типизированные события:

ItemAdded

ItemRemoved

ResourceMined

PlanetEntered

PlanetExited

SystemDiscovered

QuestAccepted

QuestCompleted

ShipDamaged

BaseModulePlaced

SaveRequested

Godot Signals использовать для локального взаимодействия сцены.

Доменный event bus использовать для бизнес-логики.

39. Этапы разработки

Этап 0. Технические прототипы

До начала основной игры должны быть завершены пять независимых прототипов.

Прототип А. Персонаж

- плоская сцена;
- управление;

- камера;
- прыжок;
- взаимодействие;
- простая стрельба.

Прототип В. Чанк рельефа

- noise;
- mesh;
- collision;
- LOD;
- фоновая генерация;
- выгрузка.

Прототип С. Сферическая планета

- cube sphere;
- гравитация к центру;
- ходьба;
- floating origin;
- швы LOD.

Прототип D. Корабль

- полёт;
- посадка;
- взлёт;
- переход к поверхности;
- переключение камер.

Прототип E. Сохранение

- SQLite;
- инвентарь;
- позиция;
- корабль;
- посещённая планета;
- резервная копия;
- восстановление.

Переход к основной разработке разрешён только после принятия всех пяти прототипов.

Этап 1. Вертикальный срез

Содержание:

- одна тестовая система;

- одна планета;
- один биом;
- персонаж;
- корабль;
- добыча;
- 10 ресурсов;
- 10 рецептов;
- одна станция;
- торговля;
- сохранение;
- один NPC;
- три задания.

Критерий:

Игрок появляется на планете, добывает ресурсы, ремонтирует корабль, взлетает, посещает станцию и возвращается.

Этап 2. Одна полноценная система

- 3–5 планет;
- разные биомы;
- фауна;
- вода;
- атмосфера;
- строительство;
- экономика;
- фракция;
- 20 заданий;
- каталог открытий.

Этап 3. Процедурная галактика

- карта;
- генерация систем;
- гиперпереходы;
- несколько типов звёзд;
- экономика систем;
- сохранение открытий;
- маршрутизация.

Этап 4. Контентная альфа

- все типы планет;

- три фракции;
- корабли;
- технологии;
- базы;
- сюжет;
- оптимизация;
- Windows/Linux.

Этап 5. Версия 1.0

- завершённый игровой цикл;
 - стабильные сохранения;
 - локализация;
 - обучение;
 - настройки;
 - оптимизация;
 - тестирование;
 - сборки;
 - облачные сохранения как необязательная функция.
-

40. Оценка трудоёмкости

При двух начинающих разработчиках:

- прототипы: 6–12 месяцев;
- вертикальный срез: ещё 6–12 месяцев;
- рабочая альфа: 2–3 года;
- версия 1.0: 3–5 лет и более.

При команде из 5–7 специалистов:

- прототипы: 3–5 месяцев;
- вертикальный срез: 6–9 месяцев;
- альфа: 12–18 месяцев;
- версия 1.0: 18–30 месяцев.

Минимальные роли:

- ведущий программист;
- gameplay-программист;
- технический 3D-художник;
- 3D-художник/аниматор;

- геймдизайнер;
 - UI/UX;
 - звук и музыка частично на аутсорсе;
 - тестирование частично совмещается до альфы.
-

41. Критерии приёмки версии 1.0

Версия считается готовой, если одновременно выполняются условия:

1. Игра запускается на Windows 10/11 и Linux.
 2. Один seed создаёт одинаковый базовый мир на обеих платформах.
 3. Игрок может посетить не менее 100 процедурных систем без ручного контента для каждой.
 4. Доступны посадка и взлёт.
 5. Доступен межзвёздный переход.
 6. Работают добыча, крафт, торговля и улучшения.
 7. Работают NPC, фракции и задания.
 8. Работают фауна и флора.
 9. Работает строительство базы.
 10. Работают сохранение, баскир и миграции.
 11. Одиночная игра полностью работает без интернета.
 12. На минимальной конфигурации достигается 30 FPS на Low.
 13. На рекомендуемой конфигурации достигается 60 FPS на Medium.
 14. Нет потери сохранения при штатном выходе.
 15. Аварийное завершение не уничтожает последнюю исправную копию.
 16. Восьмичасовой автоматический тест не вызывает критического сбоя.
 17. Отсутствуют блокирующие задания.
 18. Все пользовательские строки локализованы.
 19. Нет нелегальных сторонних материалов.
 20. Опубликовано руководство пользователя и технический журнал версий.
-

42. Прямые запреты

В проекте запрещается:

1. писать собственный OpenGL-движок;
2. хранить всю галактику в памяти;
3. заранее генерировать все планеты;
4. сохранять полный процедурный мир в БД;
5. создавать Node для каждой травинки;

6. строить navmesh всей планеты;
 7. рассчитывать реальную небесную механику;
 8. начинать multiplayer до готовности одиночного вертикального среза;
 9. внедрять микросервисы без нагрузки;
 10. внедрять ECS только ради моды;
 11. использовать фотореализм как обязательную цель;
 12. добавлять изменение рельефа до стабильного стриминга;
 13. добавлять воксельные пещеры до стабильных планет;
 14. менять алгоритм генерации без версии;
 15. использовать автоинкрементные ID для статического контента;
 16. привязывать доменную логику к сценам Godot;
 17. выполнять генерацию или SQL в основном потоке;
 18. принимать новые механики без оценки влияния на производительность и сохранения.
-

43. Итоговая конфигурация проекта

Движок:

Godot 4.7.1 .NET

Основной язык:

C#

Рендерер:

Mobile / Vulkan

Резервный рендерер:

Compatibility / OpenGL 3.3

Локальная БД:

SQLite + Microsoft.Data.Sqlite

Статические данные:

JSON

Сервер:

ASP.NET Core + PostgreSQL + S3-compatible storage

Выделенный игровой сервер:

Godot headless

Сетевой транспорт:

ENet / UDP

3D-редактор:

Blender

Формат 3D:
glTF 2.0 / GLB

Платформы:
Windows 10/11 x64
Linux x86_64

Основной режим версии 1.0:
Одиночная игра

Масштаб:
1 процедурная галактика
до 1 048 576 потенциальных систем

Модель планеты:
Cube sphere + quadtree LOD

Координаты:
Иерархические координаты + floating origin

Принцип хранения:
Seed + изменения мира

Визуальный стиль:
Стилизованный PBR средней детализации